Selección de Modelos y Reducción de Dimensionalidad

Caso de aplicación — Sesiones 31 y 32

Contexto Estratégico y de Negocio

En la Sesión 30 nos enfocamos en dejar los datos en condiciones confiables antes de modelar. En las Sesiones 31 y 32 dimos el paso siguiente: entender cómo funciona el aprendizaje automático y cómo preparar las variables para que un modelo realmente pueda aprovecharlas.

Este caso integra ambas sesiones: primero hay que decidir qué tipo de enfoque de Machine Learning corresponde a cada pregunta de negocio (Sesión 31), y luego —una vez que se decide entrenar un modelo— hay que resolver cuántas variables usar y cómo representarlas antes de dárselas al algoritmo.

A diferencia del caso de la Sesión 30 (que usaba datos ficticios de un banco), aquí trabajamos con la base real de clientes de telco (churn.xlsx) encontrarás en los materiales del curso. Todos los números y correlaciones que aparecen en este documento corresponden exactamente a esos datos reales, así que puedes verificarlos tú mismo(a) cuando lleves a cabo la actividad.

Contexto del problema:

Una empresa de telecomunicaciones cuenta con información de 3.150 clientes y 12 variables de comportamiento**:** número de fallas técnicas, reclamos, antigüedad, monto del plan, segundos y frecuencia de llamadas, SMS, contactos distintos, tipo de plan, estado de la cuenta, edad y valor del cliente o CLV.

Históricamente, el **15,7%** de los clientes ha cancelado su suscripción (fuga). Dos equipos de la empresa llegan con solicitudes distintas al mismo tiempo, y el equipo de datos también enfrenta un problema propio con el número creciente de variables disponibles.

Desafío 1: Elegir el paradigma de aprendizaje correcto

El equipo de Retención quiere anticipar, para cada cliente, si va a cancelar su suscripción el próximo mes. Cuentan con el historial completo: para cada cliente del pasado, ya se sabe si finalmente canceló o no (la columna fuga, con 0 o 1).

Al mismo tiempo, el equipo de Marketing pide algo distinto: quiere descubrir “tipos” naturales de clientes según su comportamiento de uso, sin partir de ninguna etiqueta predefinida, para diseñar campañas distintas para cada grupo.

Aplica lo aprendido con las siguientes preguntas.

Pregunta 1.1: ¿Cuál de las dos iniciativas corresponde a aprendizaje supervisado y cuál a aprendizaje no supervisado? Justifica usando el criterio de tipo de datos visto en clase (etiquetados vs. no etiquetados).

> **Respuesta**
>
> La iniciativa de Retención corresponde a aprendizaje supervisado, porque el equipo cuenta con datos etiquetados: para cada cliente histórico ya se conoce la respuesta correcta (fuga = 0 o 1). El modelo aprende a partir de esos ejemplos ya resueltos.
>
> La iniciativa de Marketing corresponde a aprendizaje no supervisado, porque no existe ninguna etiqueta previa que indique a qué “tipo” pertenece cada cliente. El objetivo es descubrir patrones o agrupaciones ocultas únicamente a partir de las variables de comportamiento, sin que nadie le diga al algoritmo qué buscar.

Pregunta 1.2: Para la iniciativa de Retención, ¿es un problema de clasificación o de regresión? ¿Cuál sería la variable objetivo (y)? Si en cambio quisiéramos predecir "cuántos meses le quedan de suscripción al cliente antes de cancelar", ¿cambiaría el tipo de problema?

Respuesta

Es un problema de clasificación, porque la variable objetivo (fuga) es una categoría discreta (0 = se queda, 1 = cancela), no un valor numérico continuo. La variable objetivo y sería, entonces, fuga.

Si en cambio se quisiera predecir "cuántos meses le quedan al cliente antes de cancelar", el problema cambiaría a uno de regresión, porque la variable a predecir pasaría a ser un número continuo (meses), no una categoría. El algoritmo también cambiaría: de una Regresión Logística a, por ejemplo, una Regresión Lineal.

Desafío 2: El problema de la alta dimensionalidad y la representación de variables

El equipo de datos, antes de entrenar cualquier modelo, revisa la matriz de correlación de las 12 variables originales y encuentra varios pares con relaciones muy fuertes:

Par de variables	Correlación	Lectura rápida
segundos ↔ frec_llamadas	0.90	Casi la misma información: más llamadas, casi siempre más segundos hablados.
frec_sms ↔ clv	0.92	El valor del cliente está fuertemente ligado a cuánto envía SMS.
n_contactos ↔ frec_llamadas	0.73	Los clientes que llaman más también contactan a más números distintos.
numero_fallas ↔ monto	0.59	Los clientes con planes de mayor monto reportan más fallas técnicas.
estado ↔ segundos	−0.30	Los clientes inactivos (estado = 2) tienden a usar mucho menos el servicio.

Además, el equipo confirma que tipo_plan y estado están codificadas como enteros (1 y 2), y que segundos llega a miles mientras que reclamos solo toma valores 0 o 1.

Aplica lo aprendido con las siguientes preguntas.

Pregunta 2.1: ¿Qué problema de los vistos en clase están enfrentando con estas correlaciones tan altas (0.90, 0.92, 0.73)? ¿Qué riesgo trae para el modelo si se dejan todas estas variables tal cual, sin hacer nada al respecto?

Respuesta

El problema es multicolinealidad: varias variables (segundos, frec_llamadas, n_contactos, frec_sms, clv) están tan correlacionadas entre sí que, en la práctica, están contando casi la misma historia ("cuánto usa el cliente el servicio") con distintas palabras.

El riesgo es que estas variables generan inestabilidad e ineficiencia en modelos como la regresión logística, además de aumentar innecesariamente el costo computacional y dificultar la interpretación del modelo, sin aportar información realmente nueva a cambio de esas columnas extra.

Pregunta 2.2: El equipo decide aplicar PCA para resumir estas variables. Antes de poder hacerlo, deben resolver dos pasos de representación de variables vistos en la Sesión 32. ¿Cuáles son esos dos pasos y por qué son obligatorios antes de aplicar PCA?

> **Respuesta**
>
> El primer paso es la estandarización (escalamiento) de las variables numéricas. Es obligatorio porque PCA trabaja con varianzas y distancias: si segundos (que llega a miles) y reclamos (que es 0 o 1) se dejan en su escala original, la variable de mayor magnitud dominaría por completo los componentes, sin que eso refleje realmente su importancia.
>
> El segundo paso es la codificación de las variables categóricas (tipo_plan y estado). Como estas variables solo están codificadas como enteros (1 y 2), PCA interpretaría esa diferencia como una distancia numérica real, cuando en realidad son categorías sin ese significado. Por eso conviene transformarlas con one-hot encoding antes de aplicar PCA.

Desafío 3: Aplicar PCA e interpretar sus resultados

Con las variables ya representadas (15 columnas en total, después de escalar, codificar y agregar 3 variables construidas por feature engineering), el equipo corre PCA sobre los datos reales y obtiene el siguiente resultado:

Modelo	N° de variables	F1-score	MCC
Variables originales	15	0.574	0.546
7 componentes (PCA)	7	0.548	0.534

El scree plot muestra que se necesitan 7 componentes para cubrir el 80,4% de la varianza total (el primer componente, PC1, explica por sí solo el 26,3%). Además, al revisar el círculo de correlaciones, el equipo nota que numero_fallas y reclamos apuntan en una dirección casi opuesta al grupo de variables de uso (segundos, frec_llamadas, monto, clv).

Aplica lo aprendido con las siguientes preguntas.

Pregunta 3.1: Con base en la tabla anterior, ¿le recomendarías al equipo usar los 7 componentes de PCA en vez de las 15 variables originales para el modelo final? Argumenta con el trade-off de desempeño visto en clase, es decir, qué se gana y qué se pierde al reducir el número de variables.

> **Respuesta**
>
> En general, sí podría valer la pena. Al pasar de 15 variables a 7 componentes, el modelo utiliza menos de la mitad de las variables, mientras que su desempeño disminuye relativamente poco.
>
> Esto muestra que **reducir dimensiones tiene un costo y un beneficio**: podemos perder algo de capacidad predictiva, pero obtenemos un modelo más simple, más liviano y más fácil de mantener.
>
> La decisión final depende del objetivo del negocio: si esa pequeña pérdida de desempeño es aceptable a cambio de simplificar el modelo, usar PCA puede ser una buena decisión. Si necesitamos obtener el máximo desempeño posible, podría convenir mantener las variables originales.

Pregunta 3.2: En el círculo de correlaciones, *numero_fallas* y *reclamos* apuntan en una dirección casi opuesta al grupo de variables de uso del servicio. ¿Qué interpretación de negocio le darías a este patrón, y cómo podría usarlo el equipo de Retención?

Respuesta

Este patrón sugiere que los clientes con más fallas técnicas y reclamos tienden a ser, en promedio, los que menos usan el servicio — posiblemente porque la insatisfacción reduce su actividad antes incluso de que decidan cancelar formalmente.

El equipo de Retención podría usar esta señal como una alerta temprana: un cliente que empieza a acumular fallas y reclamos, y al mismo tiempo reduce su uso, podría estar en las etapas iniciales de un proceso de fuga silenciosa, incluso antes de que su antigüedad o su historial lo sugieran directamente.

Desafío 4: t-SNE, visualización y la decisión final de producción

Un colega del equipo de datos queda impresionado con los mapas de t-SNE generados sobre este mismo dataset y propone reemplazar PCA por t-SNE como paso estándar de reducción de dimensionalidad antes de entrenar el modelo que va a producción.

Además, al correr t-SNE, el equipo prueba dos valores de perplexity muy distintos (5 y 80) sobre los mismos datos, y observa que los mapas resultantes se ven notoriamente diferentes entre sí.

Aplica lo aprendido con las siguientes preguntas.

Pregunta 4.1: ¿Es una buena idea reemplazar PCA por t-SNE como paso de reducción de dimensionalidad antes de entrenar el modelo que va a producción? Justifica con al menos dos diferencias entre PCA y t-SNE vistas en clase.

Respuesta

No es una buena idea. t-SNE está pensado para explorar y visualizar la estructura de los datos, no para producir variables reutilizables en un pipeline de producción.

Dos diferencias clave lo explican: **primero**, t-SNE no tiene una función equivalente a **transform()** que permita proyectar clientes nuevos usando lo aprendido con los datos de entrenamiento — cada vez que llegan datos nuevos, habría que volver a correr todo el algoritmo desde cero, lo que no es práctico en producción.

Segundo, t-SNE no es determinístico: cada corrida puede dar un resultado levemente distinto, mientras que PCA siempre entrega exactamente el mismo resultado, algo esencial para un modelo que debe comportarse de forma consistente en producción.

Pregunta 4.2: Si se corre t-SNE con perplexity=5 y con perplexity=80 sobre esta misma base, ¿qué diferencia esperarías ver en la forma de los grupos resultantes? ¿Cuál usarías para presentar un mapa general de "tipos de cliente" a la gerencia, y cuál para un análisis más detallado de subgrupos pequeños?

Respuesta

Con perplexity=5, el mapa tiende a mostrarse "desmenuzado" en muchos grupos pequeños y muy locales, porque cada punto solo le presta atención a sus 5 vecinos más cercanos. Con perplexity=80, el mapa se ve más consolidado y panorámico, con grupos más grandes y algo más de mezcla dentro de cada uno, porque cada punto considera una vecindad mucho más amplia.

Para presentar un mapa general a la gerencia conviene usar una perplexity más alta (por ejemplo 80), porque entrega una lectura más global y fácil de resumir en pocas categorías. Para un análisis más detallado, orientado a encontrar subgrupos pequeños y específicos (por ejemplo, un nicho de clientes con un comportamiento muy particular), conviene usar una perplexity baja (por ejemplo 5), que preserva mejor esa estructura local fina.